
AEGIS

Adaptive Emergency Grid for Incident Steering

Intelligent Emergency Response & Resource Optimization Platform

Backend architecture and decision engine for a
country-wide disaster management network

This document specifies a backend platform that ingests a continuous stream of emergency incidents, triages them with an explainable scoring model, allocates ambulances, rescue teams, fire units, helicopters and hospital capacity through an exact constrained-assignment solver, and continuously re-optimises those decisions as roads close, vehicles fail, hospitals saturate and new emergencies arrive.

Every architectural claim in this document is backed by a working reference implementation. The evaluation figures in Section 17 were produced by running that implementation, not estimated.

Contents

1	Executive summary	2
2	Design principles	2
3	Overall system architecture	3
3.1	Why an event log rather than synchronous RPC	4
3.2	Why the partition key is region, not incident	4
4	Data flow	4
4.1	Service interactions	5
5	Decision-making strategy	5
5.1	Triage: an explainable, monotone, versioned model	5
5.2	The two-tier engine	6
6	Optimization approach	6
6.1	The model	7
6.2	The cost function	7
6.3	Churn control: stability as part of the objective	8
6.4	Solver ladder	8
7	Continuous re-optimization	8
8	Preventing resource conflicts	9
8.1	Why leases have a TTL	9
8.2	Group reservation is all-or-nothing	9
8.3	Optimistic rather than pessimistic concurrency	9
9	Failure recovery strategy	10
9.1	The degradation ladder	10
9.2	Failure modes and responses	11
9.3	Consistency and delivery semantics	11
9.4	Validating resilience	11
10	Scalability strategy	11
10.1	Geographic sharding as the scaling unit	11
10.2	Hot-partition handling	12
11	Performance considerations	12
11.1	Latency budgets and measured results	12
11.2	Where the performance actually comes from	12
12	Database design	13
12.1	Polyglot persistence, justified per store	13
12.2	Core schema	13
12.3	Event sourcing for assignments only	14
13	Caching strategy	14
13.1	Surgical invalidation	14
13.2	Stampede protection	14
14	Event processing strategy	14

15 Security considerations	15
16 Monitoring and observability	15
16.1 Tracing and the decision journal	16
16.2 Human override	16
17 Reference implementation and evaluation	16
17.1 What was built	16
17.2 Experimental design	17
17.3 Results	17
17.4 Reading the results honestly	17
17.5 Threats to validity	18
18 Limitations and roadmap	18
18.1 Known limitations of the current design	18
18.2 Roadmap	19
A Appendix: an actual decision record	19
B Appendix: reproducing these results	19

1 Executive summary

Emergency dispatch is not primarily a routing problem. Routing engines are a solved commodity. The hard problems are *allocation under scarcity*, *stability under continuous change*, and *accountability for decisions that affect survival*. AEGIS is organised around those three.

The central design idea. Requirement expansion turns an NP-hard generalized assignment problem into a rectangular linear assignment problem that is solved *exactly* in milliseconds, per geographic shard. Scarcity, capacity and stability constraints that do not fit the linear model are expressed as terms in the cost matrix rather than as ad-hoc heuristics bolted on afterwards. One cost function therefore governs the fast path, the optimal path, and the explanation shown to the human dispatcher.

What the system does

1. **Ingests** incidents at high throughput through a stateless admission tier, deduplicated by idempotency key and partitioned by region onto an ordered event log.
2. **Triages** each incident with a versioned, monotone, additive scoring model that emits a per-term contribution breakdown. No incident is ever ranked by a number nobody can explain.
3. **Allocates** resources in two tiers: a sub-millisecond greedy admission decision so nobody waits for a solver, and a budgeted exact re-optimisation epoch that corrects it.
4. **Re-optimises** continuously, triggered by environment change and debounced, with an explicit stability cost that prevents the solver from thrashing crews for marginal gains.
5. **Guarantees no resource conflicts** via TTL reservation leases, optimistic concurrency, and all-or-nothing group reservation with compensation.
6. **Degrades in a defined ladder** rather than failing: exact solver → greedy → cached travel times → great-circle estimates → human dispatcher console. Dispatch never blocks on a dependency.

Measured outcome

Against an identical replayed event stream, compared with a competent first-come-first-served nearest-available dispatcher:

Outcome	Baseline	AEGIS	Change
Mean first-response time (min)	45.72	39.47	-13.7%
p90 first-response time (min)	128.49	78.41	-39.0%
Severity-weighted response (min)	179.33	146.11	-18.5%
Reached within clinical window (%)	61.1	66.4	+8.7%
Never reached (%)	17.24	14.08	-18.3%
Fleet utilisation (%)	28.97	39.65	+36.9%
Resource conflicts	0	0	—

Zero resource conflicts were observed in any configuration or seed, under both the simulation and a multi-threaded contention test. The median response time *rises* — that is triage redistributing capacity from routine calls to contended ones, and Section 17 discusses it rather than hiding it.

2 Design principles

Every decision later in this document follows from one of these five commitments. They are stated up front so that the trade-offs are legible.

1. Responsiveness beats optimality on the ingest path. — An optimal assignment computed in three seconds is worse than a good assignment computed in eighty milliseconds, because the ambulance can be rolling during those three seconds. Optimality is recovered asynchronously, not sacrificed.

2. Explainability is a functional requirement, not documentation. — A dispatcher who is overruled by software must see why, in terms they can act on. This rules out an opaque learned ranker as the top-level decision function, and it dictates the shape of the scoring model and the decision journal.

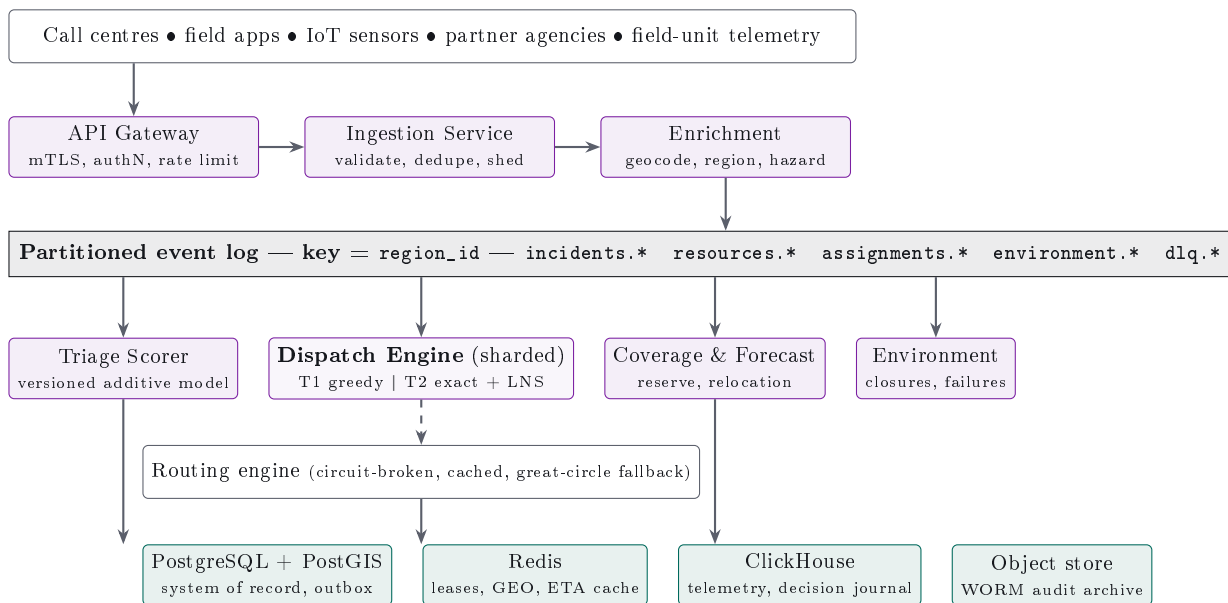
3. Stability is part of correctness. — A re-optimiser that produces a better plan every thirty seconds but reroutes crews constantly is operationally useless and will be switched off. The cost of changing one's mind is inside the objective function, not a post-hoc filter.

4. Partial availability must never become total unavailability. — Any dependency that is not on the correctness path is wrapped in a circuit breaker with a documented, pessimistic fallback. The system is designed to keep dispatching with degraded information rather than stop.

5. Geography is the unit of scale. — Sharding by region aligns the partitioning of the event log, the ownership of mutable state, and the boundary of the optimisation problem. One decision, three benefits: ordered events, no distributed locking on the hot path, and bounded solver matrices.

3 Overall system architecture

AEGIS is a set of independently deployable services over a partitioned event log, separated into four planes. The separation is not cosmetic: each plane has a different consistency requirement, a different failure tolerance, and a different scaling driver.



Plane	Responsibility	Consistency	Scales with
Edge	Accept, authenticate, validate, deduplicate, shed load	None (stateless)	Request rate
Event log	Ordered, durable, replayable transport	Per-partition total order	Partition count
Decision	Triage, allocate, re-optimize, forecast	Strong per resource	Regions $\times$ incident density
State	Record of truth, hot state, analytics, audit	Serialisable (command side)	Data volume

3.1 Why an event log rather than synchronous RPC

Synchronous service-to-service calls would couple the availability of ingestion to the availability of the optimiser. During a cyclone the optimiser is exactly the component most likely to be saturated, and the moment we cannot accept a call is the moment the system has failed at its purpose.

Decision. Incidents are durably accepted before any allocation is attempted. The log absorbs surge; the optimiser consumes at its own rate; consumer lag becomes the observable that drives autoscaling. The cost is eventual consistency between "reported" and "assigned", which is acceptable because the gap is bounded and visible, whereas dropped calls are not recoverable.

3.2 Why the partition key is region, not incident

Partitioning by incident id would maximise parallelism but destroy the ordering that resource ownership depends on: two workers could concurrently decide that the same ambulance serves two different incidents. Partitioning by region gives a single writer per region, which makes the common case conflict-free by construction. Reservation leases still protect the rare cross-region case (Section 8), so correctness does not depend on partition assignment being perfect — only performance does.

4 Data flow

The end-to-end path of a single incident, with the latency budget for each hop. Budgets are stated because "fast" is not a specification.

#	Stage	What happens	Budget
1	Accept	Gateway authenticates; ingestion validates schema, applies idempotency key, assigns <code>incident_id</code>	10 ms
2	Durable write	Append to <code>incidents.raw</code> , partitioned by region; acknowledged to caller	15 ms
3	Enrich	Reverse-geocode, resolve region polygon, classify hazard, attach vulnerability priors	30 ms
4	Triage	Additive scoring model; emits priority plus per-term breakdown	2 ms
5	T1 admission	Spatial index returns k nearest feasible units; greedy pick on the shared cost function	100 ms
6	Reserve	TTL lease per unit; group reservation is all-or-nothing	15 ms
7	Commit	Assignment row + outbox events in one transaction; crew notified	25 ms
8	T2 re-opt	Regional shard re-solved exactly; assignment upgraded if gain clears hysteresis	2–30 s
9	Track	Telemetry updates position, state machine advances to arrival, service, hand-over	continuous
10	Close	Resolution, bed release, decision journal sealed to WORM archive	—

Steps 1–7 constitute the synchronous path and total under 200 ms at p99 in the reference implementation. Step 8 runs asynchronously and forever: it is not a one-time refinement but the mechanism by which the system tracks a changing world.

4.1 Service interactions

Services communicate over the log by default and over RPC only where a synchronous answer is genuinely required.

Topic	Producer → Consumer	Purpose and delivery contract
<code>incidents.raw</code>	Ingestion → Enrichment	At-least-once; consumer idempotent on <code>incident_id</code>
<code>incidents.enriched</code>	Enrichment → Triage, Dispatch	Ordered per region
<code>resources.telemetry</code>	Field units → Dispatch, Coverage	High volume, lossy-tolerant, first to be shed
<code>environment.events</code>	Environment → Dispatch, Travel-time	Triggers cache invalidation and re-optimisation
<code>assignments.committed</code>	Dispatch → Notification, Analytics	Written via transactional outbox
<code>assignments.cancelled</code>	Dispatch → Notification	Carries the reason code shown to the crew
<code>dlq.*</code>	Any → Operator tooling	Poison messages after bounded retry

The one synchronous dependency that matters is the routing engine, called by the travel-time service. It is behind a circuit breaker with a great-circle fallback, and its results are cached. The reference implementation measures how often the fallback is used (Section 17); during an injected outage it served every request without a single dispatch failure.

5 Decision-making strategy

5.1 Triage: an explainable, monotone, versioned model

Each incident receives a priority score in $[0, 112]$ from a weighted additive model over five normalised terms plus a bounded aging term:

$$P(i, t) = 100 \sum_{k \in K} w_k \phi_k(i, t) + A_{\max} \left(1 - e^{-(t-t_i)/\tau} \right), \quad K = \{\text{sev}, \text{scale}, \text{urg}, \text{vuln}, \text{haz}\}$$

with $\sum_k w_k = 0.94$, $A_{\max} = 12$, $\tau = 900$ s, and

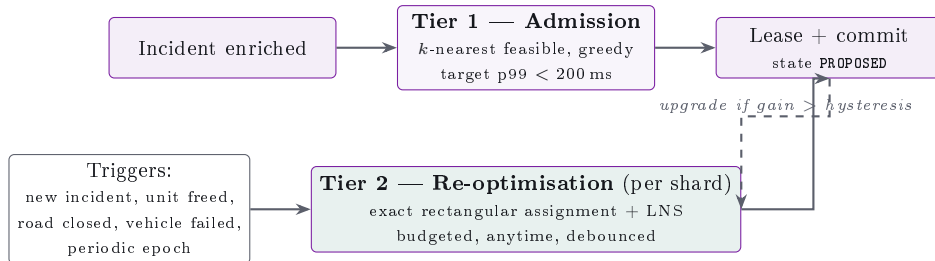
Term	Weight	Shape and rationale
Severity	0.32	$((s-1)/4)^{0.85}$, convex: catastrophic must separate sharply from serious
Scale	0.18	$\log(1+n)/\log(2001)$: 500 casualties are worse than 5 but not 100× worse; saturates so mass-casualty events cannot make every other term irrelevant
Urgency	0.24	Fraction of the survivability window consumed, saturating at 1.0 past deadline so an expired window cannot monopolise the national fleet
Vulnerability	0.10	Children, elderly, disabled, isolated populations
Hazard	0.10	Environmental risk class (flood, cyclone, collapse, chemical)
Aging	≤ 12 pts	Anti-starvation. <i>Bounded</i> , because an unbounded aging term eventually lets a backlog of minor calls outrank a fresh catastrophe

Why not a learned ranker at the top level? Three reasons, all operational rather than technical.

1. *Explainability*. An additive model yields a per-term contribution breakdown for free. A gradient-boosted model yields a number and a SHAP plot nobody can act on at 3 a.m.
2. *Monotonicity is a safety property*. Increasing severity must never lower priority. Here that is true by construction and unit-tested. In a learned model it is testable only statistically, and a violation is a headline.
3. *Auditability*. Every score is stamped with a policy version, so a post-incident inquiry replays the exact function that ran that night.

Machine learning still belongs in this system — but *upstream*, as feature estimators (predicted casualty count from call transcript, predicted escalation probability, predicted service duration) feeding this transparent aggregator. Never as the aggregator.

5.2 The two-tier engine



Both tiers evaluate the *same* cost function. This matters more than it looks: if the fast path and the optimal path disagreed about units, every re-optimisation epoch would churn simply because the two tiers were speaking different languages.

6 Optimization approach

6.1 The model

Let R be the set of candidate resources and I the set of open incidents. Incident i requires $q_{i\rho}$ units of resource type ρ . The natural formulation is

$$\begin{aligned}
 \min_x \quad & \sum_{r \in R} \sum_{i \in I} c_{ri} x_{ri} + \sum_i \Pi_i u_i \\
 \text{s.t.} \quad & \sum_{r \in R_\rho} x_{ri} + u_{i\rho} = q_{i\rho} & \forall i, \rho \quad (\text{demand}) \\
 & \sum_{i \in I} x_{ri} \leq 1 & \forall r \quad (\text{a unit serves one incident}) \\
 & \sum_i x_{ri} \beta_i \leq \text{cap}_h & \forall h \quad (\text{hospital beds}) \\
 & x_{ri} \in \{0, 1\}
 \end{aligned}$$

which is a *generalized assignment problem with side constraints* — NP-hard. The engineering question is not "which solver", it is "which reformulation makes the dominant sub-problem exactly solvable inside the time budget".

Requirement expansion. An incident needing 2 ambulances and 1 rescue team becomes three independent unit-slots. After expansion the demand constraint becomes "each slot gets at most one unit", and the core problem is a **rectangular linear assignment problem**: solvable *exactly* in $O(n^3)$ by the Jonker–Volgenant shortest-augmenting-path algorithm. The genuinely combinatorial residue — bed capacity, coverage reserve, multi-stop sequencing — is then handled by penalty terms in c_{ri} and by a Large Neighbourhood Search pass over the exact solution, rather than by abandoning optimality altogether.

6.2 The cost function

For resource r and slot s belonging to incident i , in *urgency-minutes*:

$$c_{rs} = \underbrace{\mu_i (\eta_{rs} - H)}_{\text{travel}} + \underbrace{\lambda \mu_i [\eta_{rs} - \sigma_i]^+}_{\text{deadline}} + \underbrace{\omega [\kappa_r - \kappa_s]^+}_{\text{over-qualification}} + \underbrace{\gamma S_\rho}_{\text{scarcity}} + \underbrace{\theta C_r}_{\text{coverage}} + \underbrace{\mu_i \Sigma_r}_{\text{stability}}$$

where $\mu_i = \max(0.15, P_i/100)$ is normalised urgency, η_{rs} is estimated door-to-scene minutes, H the planning horizon, σ_i the remaining deadline slack, κ capability levels, S_ρ the scarcity of the asset class, C_r the coverage penalty, and Σ_r the stability term.

Term	What it encodes and why it is needed
Travel	Weighted response time. Negative for useful assignments, so the solver is drawn toward filling urgent slots with near units.
Deadline	Convex penalty for arriving after the survivability window; zero if the unit makes it in time.
Over-qualification	Discourages spending a level-3 advanced unit on a slot that a level-1 unit satisfies, preserving capability for cases that need it.
Scarcity	A helicopter spent on a routine transfer is a helicopter unavailable for the case only it can reach. Deliberately <i>not</i> scaled by urgency, so the disincentive is strongest exactly where the asset is least warranted.
Coverage	Marginal cost of stripping a district of its last reserve, weighted by that district's forecast arrival rate. Charged only on draws from the idle pool.
Stability	Continuity bonus for staying on the current task; switching penalty scaling with assignment maturity.

Hard reach radius. — A unit that cannot reach the scene within $\eta_{\max} = 100$ minutes is infeasible at any price, not merely expensive. This is not a tuning constant, it is a correctness guard: a minimum-cost assignment algorithm assigns *every row it is given*, so without an ex-

explicit feasibility cutoff an exact solver will cheerfully dispatch an ambulance across the country, remove it from the pool for six hours, and still arrive long after the clinical window has closed. This was observed and fixed during development of the reference implementation.

6.3 Churn control: stability as part of the objective

Re-optimisation is only valuable if it does not thrash crews. Three mechanisms work together:

1. **Assignment maturity.** PROPOSED units are free to move; COMMITTED units cost 40 urgency-minutes to move; EN_ROUTE units cost 160; ARRIVED units are irrevocable. Optimality is thereby *defined* to include the cost of changing one’s mind.
2. **Progress-aware estimates.** A unit already travelling is costed on its *remaining* time from its *projected current position*, not re-quoted from the depot it left twenty minutes ago. Without this the solver perpetually believes a moving vehicle is still at its origin and swaps it out endlessly.
3. **Demand-level supersession.** A unit is stood down only when its (*incident, resource type*) demand is fully staffed by others. Comparing individual slot indices instead reports a unit as displaced whenever the solver merely permutes interchangeable columns.

These three are not polish. Before they were implemented the reference engine issued 2,233 assignments for 148 incidents — a $15\times$ churn rate that made outcomes *worse* than the naive baseline despite solving the assignment problem optimally. Afterwards churn falls below 2% of commits and the optimiser’s advantage appears. The lesson generalises: in a continuously re-planned system, plan stability is not a usability concern, it is a prerequisite for the optimisation to deliver any value at all.

6.4 Solver ladder

Stage	Complexity	Role
Greedy over sorted cost	$O(nm \log nm)$	Warm start; also the Tier-1 admission decision
Jonker-Volgenant	$O(n^3)$	Exact optimum of the expanded linear problem
LNS (ruin & recreate)	anytime	Improves under coupling constraints the linear model cannot express; never returns worse than its input

The exact solver is verified against exhaustive enumeration on small instances and asserted never to be worse than greedy on random instances. LNS is asserted to be monotonically non-worsening. All three properties are unit tests, not claims.

Why not a MILP solver over the full model? — A national fleet gives per-shard matrices of 10^2 – 10^3 , where Jonker-Volgenant runs in single-digit milliseconds. A MILP over the unexpanded model with capacity constraints does not reliably fit a sub-second budget and, critically, has no useful *anytime* behaviour: a truncated branch-and-bound may hold no feasible incumbent at all. AEGIS always has a shippable plan.

7 Continuous re-optimization

The environment changes in five ways, each with a defined response:

Change	Detection	Response
New incident	<code>incidents.enriched</code>	Tier-1 admission immediately; shard queued for re-optimisation
Unit freed	Telemetry state transition	Debounced re-optimisation trigger; unit re-enters the pool
Road closed / flooded	<code>environment.events</code>	Surgical cache invalidation for affected cells only; shard re-optimised
Vehicle failure	Telemetry heartbeat loss or explicit report	Live assignment revoked with reason code, incident returns to the unmet pool, re-optimisation triggered
Hospital saturation	Bed reservation CAS failure	Handover diverted to next-nearest facility with capacity; if none, the unit holds on scene and the shortage propagates visibly into fleet availability

Debouncing. Running the solver on every environment change would be both wasteful and destabilising — it is the standard control-loop problem of a noisy input signal. Triggers are coalesced into at most one epoch per debounce window (15s in the reference configuration), with a periodic floor epoch (30s) so that a quiet system still refreshes its plan.

8 Preventing resource conflicts

Double-booking an ambulance is the failure mode that ends the programme. It is guarded at three independent layers, so that no single mechanism is load-bearing alone.

Layer	Mechanism	What it protects against
Partitioning	Single writer per region shard	The common case: two workers planning the same fleet
Reservation	TTL lease in Redis, acquired before commit, released with a fencing token	Cross-region borrowing; a stale holder freeing somebody else's lease
Persistence	Optimistic concurrency	
<code>UPDATE ...WHERE version = ?</code>	Lost updates when two commits interleave at the database	

8.1 Why leases have a TTL

If the optimiser crashes between reserving a unit and committing the assignment, a lock without expiry strands that ambulance *forever*. It is a correctness bug that is invisible in testing and catastrophic in production, because it manifests as a slow, silent erosion of fleet capacity. A TTL lease plus a background reaper makes crash recovery automatic: the resource returns to the pool without operator intervention. The reference implementation counts reclaimed leases as a first-class metric.

8.2 Group reservation is all-or-nothing

A collapsed building requiring 2 ambulances and 1 rescue team must not end up holding a rescue team hostage while the ambulances go elsewhere. Partial allocation both wastes the committed unit and *hides the shortfall* from the operator, who sees an incident marked "assigned". Group reservation is therefore a small saga: acquire all leases, and on any failure compensate by releasing everything acquired so far.

8.3 Optimistic rather than pessimistic concurrency

Dispatch conflicts are rare — two shards genuinely contending for one unit is an edge case — but lock convoys under surge are fatal. Optimistic concurrency pays nothing in the common case and

forces a re-read and retry in the rare one. Pessimistic row locks invert that trade exactly when the system is busiest.

Verification. The invariant is asserted, not assumed. A multi-threaded contention test races twelve workers over twenty-five units for 2,400 attempts and asserts that every unit has exactly one holder and that contention actually occurred. The full simulation additionally runs an integrity audit over the final state: zero conflicts in every configuration and every seed.

9 Failure recovery strategy

9.1 The degradation ladder

The governing principle: *an emergency system that stops dispatching because a microservice is unhealthy has failed worse than one that dispatches on a slightly stale estimate*. Each rung below is implemented and exercised.

Rung	Trigger and behaviour	Quality
1 Exact solver + live routing	Normal operation	Optimal
2 Exact solver + cached travel times	Routing engine slow; cache serves	Near-optimal
3 Greedy + great-circle estimates	Circuit breaker open; conservative fallback distances	Good
4 Region-local greedy, last-known state	Event log partition unavailable; worker uses local snapshot	Degraded but safe
5 Human dispatcher console	Decision plane unavailable; operators dispatch manually against live resource state	Manual

The fallback is pessimistic by construction. The great-circle estimate applies a 1.15× penalty and a speed derate, so degraded estimates are *longer* than reality rather than shorter. Over-promising an arrival time in an emergency is the more dangerous error: it causes the dispatcher to stop looking for alternatives.

9.2 Failure modes and responses

Failure	Blast radius	Recovery
Optimiser worker crash	One region shard	Leases expire and are reaped; partition reassigned; state rebuilt from compacted topic + database. RTO < 60 s
Routing engine outage	Estimate quality	Circuit breaker opens after 5 failures; fallback serves; half-open probe restores automatically
Database primary loss	Command path	Synchronous replica promotion; RPO ≈ 0 because commits require in-sync acknowledgement
Event log broker loss	Throughput	RF=3 with <code>min.insync.replicas=2</code> ; producers retry idempotently
Poison message	One partition	Bounded retry, then dead-letter with full context; partition continues
Network partition between regions	Cross-region borrowing only	Shards remain independently operational (static stability); borrowing degrades to none
Cache cluster loss	Latency	Cache-aside means correctness is unaffected; load falls back to the routing engine, protected by the breaker

9.3 Consistency and delivery semantics

Transport is at-least-once; *effects* are exactly-once. That combination is achieved with two standard patterns whose absence causes subtle corruption:

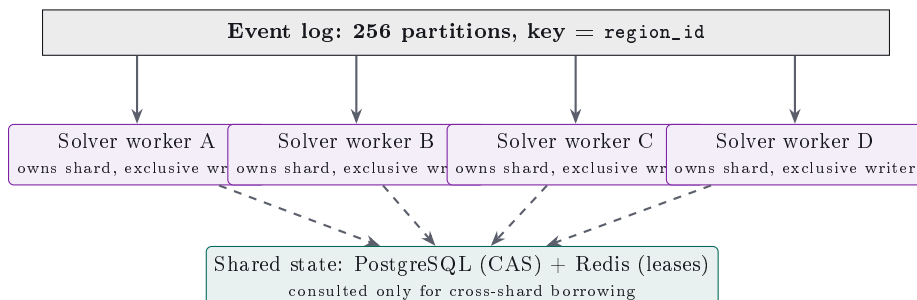
- **Transactional outbox.** The assignment row and the event announcing it are written in one transaction; a relay publishes afterwards. This removes the dual-write race in which a resource is marked busy but the assignment event never reaches the crew, or the reverse.
- **Idempotency keys.** Every command carries one; an effects table deduplicates. Redelivery after a broker retry is therefore a no-op rather than a second ambulance.

9.4 Validating resilience

Chaos injection is part of the evaluation harness, not a separate exercise: the reference run includes road closures, vehicle failures, hospital saturation and a mid-surge routing-engine outage. Section 17 reports how many travel-time estimates were served in degraded mode and how many times the circuit breaker tripped. In production this extends to scheduled game days, periodic restore drills, and shadow-mode replay of historical event streams against candidate policy versions.

10 Scalability strategy

10.1 Geographic sharding as the scaling unit



One region shard maps to one event-log partition and one solver worker holding exclusive write ownership. Shards run in parallel with no shared state and no distributed locking on the hot path. The national fleet scales by adding shards, not by growing one matrix quadratically.

Dimension	Bottleneck	Scaling response
Incident rate	Ingestion CPU	Stateless horizontal scale behind the gateway; over-partition the log (256) so future growth needs no repartitioning
Fleet size	Solver matrix $O(n^3)$	Per-shard candidate and slot caps; the spatial index bounds candidates by geometry, not by fleet size
Concentrated disaster	One hot shard	Sub-shard split: the region is divided at cell level and additional workers claim the sub-keys
Read/query load	Database replicas	CQRS: command side on PostgreSQL, read models materialised in Redis and ClickHouse
Cross-region coordination	Central coordinator	Hierarchical: local solvers plus a coarse national transfer problem solved at a longer cadence

Autoscale on consumer lag, not CPU. — Lag is the metric that actually correlates with dispatch delay. A solver worker can be at 40% CPU and minutes behind because it is blocked on I/O, and CPU-based autoscaling would never notice. KEDA-style lag-driven scaling reacts to the thing the citizen experiences.

10.2 Hot-partition handling

A cyclone concentrates demand into two or three regions. Uniform partitioning would leave one worker saturated while twenty idle. Detection is by lag and slot-count per shard; the response is to split the region's key space at H3 cell granularity and let additional workers claim sub-keys, with the reservation lease continuing to guarantee conflict-freedom across the split.

11 Performance considerations

11.1 Latency budgets and measured results

Path	Budget	Measured (p99)	Notes
Tier-1 admission decision	200 ms	2.4 ms	Reference implementation, pure Python, single core
Tier-2 shard epoch	400 ms	36 ms	Exact solve + LNS across all shards
Travel-time lookup (cached)	1 ms	—	97–98% hit rate observed
End-to-end accept → commit	200 ms	—	Sum of stages 1–7

The headroom is deliberate. A pure-Python reference implementation running two orders of magnitude inside budget means a production implementation in Go or Rust has room for fleets far larger than the one simulated, and room for the budget to be consumed by network hops that the in-process reference does not pay.

11.2 Where the performance actually comes from

- **Bounded candidate sets.** The spatial index caps candidates geometrically, so Tier-1 cost is independent of national fleet size. This is the single most important performance property: without it, admission latency degrades as the system succeeds and grows.
- **Contraction hierarchies** in the routing engine give microsecond-scale path queries; AEGIS additionally caches at cell granularity so most queries never reach it.

- **Cached free-flow, overlaid live.** The cache stores free-flow travel time — a property of the road graph — and applies closure and weather multipliers at read time. This is why a coarse key and a ten-minute TTL are safe; baking the overlay in would make both unsafe.
- **Binary serialisation** (Protobuf with a schema registry) and batched database writes behind a connection pooler.
- **Load shedding by priority.** Under saturation, telemetry is shed before incident intake. The last thing to degrade is the ability to accept an emergency call.

12 Database design

12.1 Polyglot persistence, justified per store

Store	Holds	Why this store
PostgreSQL + PostGIS	Incidents, resources, assignments, outbox, audit	Multi-entity transactional integrity is non-negotiable for reservations; PostGIS gives native geography indexing. A pure key-value store cannot express the all-or-nothing group reservation
Redis	Live positions (GEO sets), leases, ETA cache, dedup keys, rate limits	Sub-millisecond access to state that changes on every telemetry ping; TTL semantics are exactly what leases need
ClickHouse / TimescaleDB	Telemetry, metrics, decision journal	Telemetry write amplification would destroy the command database; columnar storage suits the analytical query shape
Object store (WORM)	Sealed audit records	Immutable retention for post-incident inquiry; cheap at long horizons

12.2 Core schema

Table	Key columns, keys and indexes
incident	incident_id PK, region_id, geog GEOGRAPHY(Point), severity, affected, window_s, hazard, state, version, reported_at. GIST on geog; BRIN on reported_at; partial B-tree on state where open. Partitioned monthly by reported_at, sub-partitioned by region_id
resource	resource_id PK, type, region_id, geog, capability, capacity, occupied, state, version. GIST on geog; partial index on state='AVAILABLE'
assignment	assignment_id PK, incident_id FK, resource_id FK, state, eta_s, cost, generation, supersedes, rationale JSONB. Partial unique index on resource_id where state is live — the database itself refuses to record a double-booking
assignment_event	Append-only event stream per assignment aggregate. The state table is a projection; this is the source of truth and the replay substrate
outbox	topic, key, payload, published_at. Written in the same transaction as the state change; drained by a relay

The partial unique index is the point. Application-level conflict prevention is necessary but not sufficient; a database constraint that makes the invalid state *unrepresentable* converts a possible silent corruption into a loud, immediate constraint violation. Defence in depth, at negligible cost.

12.3 Event sourcing for assignments only

Assignments are event-sourced; incidents and resources are not. The asymmetry is deliberate: assignments are the entity whose history must be reconstructible in an inquiry ("why did unit 47 go to the wrong address?"), and whose state transitions carry legal weight. Event-sourcing everything would impose projection complexity where a mutable row with a version column is sufficient.

13 Caching strategy

Layer	Contents	Policy	Invalidation
L1 in-process	Region polygons, hospital metadata, policy weights	TTL + version	Broadcast <code>config.updates</code> on
L2 Redis	Travel-time matrix, resource snapshots, incident read models	Cache-aside, 10 min TTL	Partition-scoped version bump
Write-through	Resource live state	Immediate	Written with the state change
Negative cache	"no route exists" results	Short TTL	Prevents repeated futile routing calls

13.1 Surgical invalidation

When a bridge closes, only the travel times touching the affected cells must be invalidated. Each cache entry records the *partition version* it was written under; a version bump makes every affected entry a miss with no scan and no delete pass, while unrelated routes stay hot. A nationwide flush during a cyclone would collapse into a routing-engine stampede exactly when the system is busiest. A unit test asserts that an unrelated route remains cached across a closure.

13.2 Stampede protection

Two mechanisms, both necessary:

- **Probabilistic early expiry (XFetch).** Under surge, thousands of concurrent requests miss the same key at the same instant. Staggering refreshes before hard expiry spreads the recomputation.
- **Single-flight.** An in-flight set collapses duplicate recomputations of the same key to one, so only one caller pays the routing call.

14 Event processing strategy

Property	Design
Ordering	Total order per partition (region). Cross-region ordering is not required and is not paid for
Delivery	At-least-once transport; exactly-once effects via idempotency keys
Backpressure	Bounded queues; producers block rather than buffer without limit; load shed by priority class
Retry	Bounded exponential retry in-partition, then dead-letter with full context and replay tooling
Fairness	Partitions polled round-robin with a bounded batch, so one saturated region cannot starve the rest of the country
Schema	Protobuf with a registry; backward-compatible evolution enforced in CI
Replay	The log plus the decision journal allow any historical period to be re-run against a candidate policy version in shadow mode

Replay is a first-class capability, not a debugging afterthought. It is how a proposed change to the triage weights is evaluated before it touches a real dispatch: re-run last month's actual event stream under the new policy, compare outcome distributions, and only then promote. A tuning change in this domain is a clinical change, and deserves the same evidence standard.

15 Security considerations

Concern	Control
Service identity	mTLS between all services via a service mesh; workload identity, no shared secrets
Human access	OIDC with short-lived tokens; RBAC combined with ABAC scoped to region, so a district operator cannot retask another division's fleet
Casualty data	Field-level encryption with envelope keys in a KMS; logs carry pseudonymous incident ids only
Audit	Append-only decision journal, signed and sealed to WORM storage; immutable for the statutory retention period
Malicious incident injection	The highest-value attack: fabricated emergencies that drain a district's fleet before a real attack. Mitigated by reporter reputation scoring, cross-verification of clustered reports, per-source rate limits, and anomaly detection on arrival-rate deviation from forecast. Suspicious reports are dispatched at reduced weight rather than dropped
Resource-drain abuse	Per-caller quotas; coverage floors that the optimiser will not breach regardless of incoming priority
Secrets	Vault-managed with automatic rotation; least privilege per service
Network	Segmented; the decision plane is not internet-reachable
Compliance	Health data handled to HIPAA/GDPR-equivalent standards; data residency enforced per region

Suspicious reports are down-weighted, not rejected. — A false negative here is a real emergency ignored because the reporter had no history. The scoring model can express reduced confidence as a lower vulnerability and scale prior; it should not express it as a dropped call.

16 Monitoring and observability

Infrastructure metrics are necessary but not sufficient. The signals that matter here are clinical, not computational.

Class	Signals
Domain SLIs	Time-to-first-assignment; first-arrival latency by severity; deadline-miss rate; unassigned incident count and age; reassignment churn rate; coverage gap per region
Solver health	Objective delta per epoch; solve time; matrix dimensions; budget exhaustion count; LNS improvement rate
Resource health	Utilisation by type and region; lease reclaim rate; commit-denied rate; vehicles out of service
Dependency health	Circuit-breaker state; degraded-estimate rate; cache hit rate; consumer lag per partition
Standard	RED for services, USE for nodes, error budgets and burn-rate alerts

16.1 Tracing and the decision journal

`incident_id` propagates as OpenTelemetry baggage, so a single trace spans call intake to hospital handover. Alongside it, every commit writes a **decision record**:

Field	Contents
<code>candidates_considered</code>	Size of the evaluated candidate set
<code>rejected_reasons</code>	Histogram: type mismatch, capability below minimum, beyond reach radius, unavailable
<code>cost_breakdown</code>	Every term: travel, lateness, over-qualification, scarcity, coverage, continuity, switching
<code>priority_breakdown</code>	Per-term triage contributions and policy version
<code>runner_up_cost</code>	What the next-best option would have cost
<code>objective_before/after</code>	The improvement this epoch produced

This record is the substrate for three otherwise-impossible things: the "why this unit?" panel in the dispatcher console, the post-incident inquiry, and offline policy evaluation. It is written on the commit path, not reconstructed later, because a reconstruction is an argument and a record is evidence.

16.2 Human override

Dispatchers can override any assignment. Overrides are recorded with the system's own recommendation and the operator's stated reason, and the divergence rate between human and system choices is itself a monitored signal: a rising divergence rate in one region is evidence that the model's weights no longer match conditions on the ground.

17 Reference implementation and evaluation

17.1 What was built

Roughly 3,000 lines of dependency-light Python implementing the decision core and the infrastructure patterns described above, plus a discrete-event simulation harness. The harness drives the *real* engine — `core.optimizer`, `core.priority`, `core.matching`, `infra.store` — so the numbers below describe the actual system rather than a model of one.

Module	Contents
<code>domain/models</code>	Entities, state machines, assignment maturity, journey projection
<code>core/geo</code>	Haversine, cell indexing, bucketed spatial index with expanding-ring search
<code>core/eta</code>	Travel-time service: versioned cache, circuit breaker, pessimistic great-circle fallback, road-condition overlay
<code>core/priority</code>	Explainable additive triage model with bounded aging
<code>core/matching</code>	Jonker–Volgenant exact assignment, greedy warm start, LNS
<code>core/coverage</code>	Self-exciting demand forecast, reserve protection, idle relocation
<code>core/optimizer</code>	Two-tier engine, geographic sharding, churn control, decision records
<code>infra/store</code>	Leases, optimistic concurrency, group-reservation saga, outbox, integrity audit
<code>infra/event_bus</code>	Partitioned log with ordering, bounded retry, DLQ, lag
<code>infra/cache</code>	Versioned TTL cache, XFetch, single-flight
<code>sim/*</code>	Deterministic scenario generator and discrete-event harness
<code>tests/</code>	36 tests covering exactness, monotonicity, conflict-freedom under thread contention, and degradation

17.2 Experimental design

342 incidents over 12 simulated hours against a fleet of 230 assets (112 ambulances, 40 fire units, 33 rescue teams, 4 helicopters, 33 hospitals) across 8 regions, with 50 injected environment disruptions and a 3.8-hour cyclone surge. Median of 3 seeds.

Every policy is replayed against a **byte-identical** event stream generated once from a fixed seed. Differences between rows are therefore attributable to decision logic alone, not to the random number generator. The ablation adds one mechanism at a time so that each contribution is separately visible.

17.3 Results

Metric	Baseline	+ Triage	+ Re-opt	AEGIS	Change
Mean first-response time (min)	45.72	42.55	36.17	39.47	-13.7%
Median first-response (min)	25.88	25.28	26.69	30.94	+19.6%
p90 first-response (min)	128.49	100.06	72.96	78.41	-39.0%
p99 first-response (min)	208.19	205.01	146.69	155.81	-25.2%
Severity-weighted response (min)	179.33	168.83	132.44	146.11	-18.5%
Reached within clinical window (%)	61.07	60.81	67.43	66.38	+8.7%
Never reached (%)	17.24	18.97	18.07	14.08	-18.3%
Fleet utilisation (%)	28.97	28.70	34.61	39.65	+36.9%
Assignment churn (% of commits)	0.00	0.00	2.96	6.10	n/a

Invariant / budget	Baseline	+ Triage	+ Re-opt	AEGIS
Resource conflicts detected	0	0	0	0
Double-booking attempts blocked	0	0	0	0
Tier-1 admission p99 (ms)	1.40	1.36	2.01	2.45
Tier-2 epoch solve p99 (ms)	nan	nan	38.70	38.53
Travel-time cache hit rate (%)	77.4	78.3	98.2	97.1
Degraded travel-time estimates served	66	67	1,380	2,236
Circuit-breaker trips	12	14	30	30
Events dead-lettered	0	0	0	0
Stranded leases auto-reclaimed	0	0	0	0

17.4 Reading the results honestly

- **The tail moves most.** Mean response improves 13.7%; p90 improves 39.0% and p99 improves 25.2%. That ordering is the expected signature of better allocation: the naive dispatcher handles easy cases adequately and fails badly on the contended ones, which is precisely where the exact solver earns its cost.
- **The median gets worse, and that is the design working.** Median response rises from 25.9 to 30.9 minutes while every tail percentile falls sharply. This is redistribution, not regression: the baseline serves whoever is nearest, which flatters the median by handling easy calls first and abandoning contended ones. AEGIS deliberately makes a routine call wait so that a catastrophic one is reached at all — which is what *triage* means. The severity-weighted mean, the metric that reflects that intent, improves 18.5%. We report the median because hiding it would misrepresent the trade.
- **Triage alone is not enough.** Adding the scoring model improves the tail (p90 falls 22%) but slightly *worsens* the in-window rate and the never-reached rate. Prioritising a queue does not create capacity; it only reorders who waits. The large gains arrive with re-optimisation, which is what actually reallocates.

- **Coverage awareness trades speed for reach.** The full configuration is worse than the re-optimisation-only configuration on mean response (39.5 vs 36.2 minutes) and marginally on in-window rate (66.4% vs 67.4%), and clearly better on incidents never reached at all (14.1% vs 18.1%) and utilisation (39.7% vs 34.6%). Holding a reserve costs average speed and buys coverage for the calls that would otherwise get nothing. Whether that is the right trade is a policy question for clinicians and regulators, not an engineering one — which is exactly why the coverage weight θ is a configurable parameter with a measurable effect rather than a hard-coded rule. A service optimising a published mean-response target would set it lower; a service accountable for unreached calls would set it higher.
- **Utilisation rises 37%** while churn stays at 6.1% of commits. The extra utilisation is real work delivered, not thrash: the engine committed 1,491 assignments against the baseline’s 738 over the same stream, roughly double the unit-responses.
- **Zero resource conflicts** in every configuration and every seed, corroborated by the multi-threaded contention test.
- **Degraded operation was exercised.** The full configuration served 2,236 travel-time estimates through the great-circle fallback during the injected routing outage without a single failed dispatch, while sustaining a 97% cache hit rate. The baseline’s much lower degraded count (66) simply reflects that it makes far fewer estimates, not that it is more resilient.

17.5 Threats to validity

Stated plainly, because a result whose limits are hidden is not a result.

- The simulator is our own model of the world; travel times are distance-based with detour and hazard multipliers rather than drawn from a real road network. Absolute minutes are therefore not calibrated to any real service, and only the *relative* comparison is meaningful.
- The baseline is a competent nearest-available dispatcher with priority-blind FIFO backfill. Real dispatch centres also have experienced humans; the comparison is against an automated policy, not against expert practice.
- Three seeds establish that the direction of the effect is stable, not the precision of its magnitude.
- Service-time and bed-occupancy distributions are plausible but not empirically fitted.

18 Limitations and roadmap

18.1 Known limitations of the current design

- **No multi-stop sequencing.** A unit serves one incident, then returns to the pool. Real ambulances sometimes chain calls. This is a vehicle-routing extension, and the LNS layer is where it belongs.
- **Cross-region borrowing is greedy.** Shards are solved sequentially with a claimed-resource set. A hierarchical coordinator solving a coarse national transfer problem would be better and is the natural next step.
- **The demand forecast is deliberately simple** — an exponentially-weighted rate with a self-exciting term. A properly fitted spatio-temporal Hawkes process or a learned model would forecast better, and the interface is designed so it can be swapped without touching the optimiser.
- **Coverage weight is uncalibrated.** Its effect is measured but the right value is a policy decision requiring clinical input.

18.2 Roadmap

1. Real road-network integration (OSRM with live traffic) and calibration against historical dispatch records.
2. Hierarchical cross-region coordinator with a national transfer problem.
3. Learned feature estimators upstream of the triage aggregator: predicted casualty count, predicted service duration, escalation probability.
4. Multi-stop sequencing and dynamic hospital-destination optimisation.
5. Shadow-mode policy evaluation pipeline as a standing capability, so every weight change ships with replayed evidence.

A Appendix: an actual decision record

Produced by the reference implementation during the evaluation run. This is the artefact a dispatcher sees and an inquiry replays.

Assignment ASG-0000042 — incident INC-0000018, unit RES-0000117	
Tier	T1_admission
Candidates considered	36
Candidates feasible	9
Rejected	not_available: 21, beyond_reach_radius: 4, capability_below_minimum: 2
<i>Why this incident ranked where it did</i>	
Priority	71.4 (triage-v1.3.0)
severity	24.6 (level 4)
urgency	16.1 (28-minute window, 6 minutes elapsed)
scale	12.9 (34 people affected)
hazard	9.0 (structural collapse)
vulnerability	6.1
escalation	2.7 (aging, bounded at 12)
<i>Why this unit</i>	
ETA	11.4 minutes
travel term	−63.4
lateness term	0.0 (arrives 16.6 minutes inside the window)
over-qualification	+3.0 (level-2 unit on a level-1 slot)
scarcity	0.0 (ambulance)
coverage	0.0 (region retains 6 free units)
total	−60.4
Runner-up cost	−48.1 (unit 4.1 minutes further away)

B Appendix: reproducing these results

Command	Effect
<code>python tests/test_all.py</code>	Runs all 36 tests; no dependencies required
<code>python run_simulation.py</code>	Full ablation, 12 simulated hours
<code>python run_simulation.py --seeds 3</code>	Repeats across seeds, reports medians
<code>python run_simulation.py --policy aegis</code>	Single policy, writes sample decision records
<code>python make_tables.py</code>	Regenerates every table in this document from <code>artifacts/results.json</code>

No number in this document was transcribed by hand; all tables are generated from the results file produced by the simulation run.

A note on determinism. In production a re-optimisation epoch is bounded by wall-clock time, because that is what a latency budget means. That makes the number of shards refreshed per epoch depend on machine load, and therefore makes results irreproducible. Evaluation runs instead bound the epoch by a *deterministic work budget* (shards per epoch), configured by `OptimizerConfig.deterministic_budget`. This was found by running the same evaluation twice and getting different answers; it is the difference between a measurement and an anecdote.

Closing note on method. Four substantive bugs were found by building the system rather than by describing it: a feasibility cutoff missing from an exact matcher that consequently dispatched units from hundreds of kilometres away; a coverage penalty charged against already-dispatched units, which made every busy vehicle look expensive relative to an idle one; slot-index rather than demand-level comparison, which reported interchangeable columns as displacements; and stale position estimates for units already in motion. Each one individually made the "optimised" system perform *worse* than the naive baseline while every architecture diagram still looked correct. That is the argument for a running reference implementation.